# TLS / SSL – Secure Communication

## Problem Before Encryption

In older networks:

- Many protocols sent data in plaintext
- Attackers could sniff network traffic

Attack method:
Network card → Promiscuous mode → Capture all packets

Tools attackers used:

- Wireshark
- tcpdump
- Packet sniffers

What attackers could see:

- Chat messages
- Emails
- Usernames
- Passwords

Users could not prevent this because the protocol itself was insecure.

## Solution: SSL and TLS

To solve this problem, secure communication protocols were created.

SSL → First secure web protocol
TLS → Improved version of SSL

- TLS protects communication using cryptography.
- Without TLS attacker can read packets
- With TLS packets are encrypted

## Protocols That Use TLS

Many insecure protocols become secure when TLS is added.

| | |
|---|---|
| HTTP | HTTPS |
| DNS | DoT (DNS over TLS) |
| MQTT | MQTTS |
| SIP | SIPS |
| SMTP | SMTPS |
| POP3 | POP3S |
| IMAP | IMAPS |

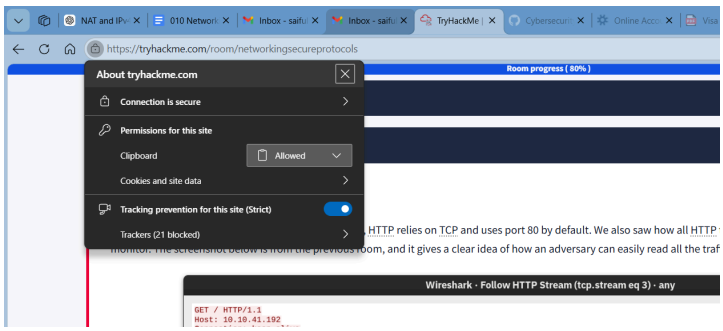
Meaning:
"S" = Secure (TLS/SSL encryption)

# TLS Certificates

For TLS to work, servers must prove their identity.

They use digital certificates.

1. Server creates CSR (Certificate Signing Request)
2. Sends CSR to Certificate Authority (CA)
3. CA verifies request
4. CA signs and issues certificate
5. Server installs certificate



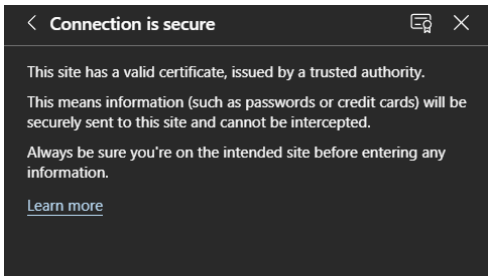

Now clients can trust the server.

## Certificate Authorities (CA)

A Certificate Authority is a trusted organization that signs certificates.

**Examples:**

- Let's Encrypt
- DigiCert
- GlobalSign
- Comodo




Browsers already contain a list of trusted CAs.

**When you visit a website:**
Browser checks → Certificate signature → Trusted CA
If valid → secure connection established.

## Self-Signed Certificates

Some servers use self-signed certificates
Meaning the Server signs its own certificate

- No trusted third party verified it
- Browser shows security warning

**Example warning:**

Your connection is not private

**Self-signed certificates are often used in:**

- Testing labs
- Internal networks
- Development environments

# HTTP and HTTPS

## HTTP

HTTP (Hypertext Transfer Protocol) is used to transfer web pages between a client and a server.

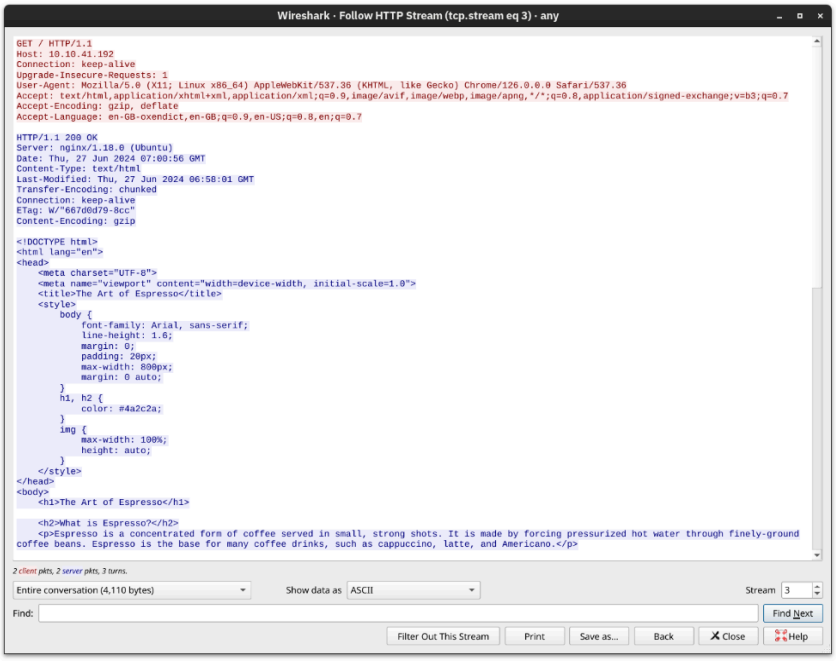
Transport Protocol → TCP
Default Port → 80
Encryption → None (plaintext)

**Example HTTP request:**

```
GET / HTTP/1.1
Host: example.com
```

**Attackers using packet sniffers can see:**

- URLs
- Login forms
- Cookies
- Session IDs
- Credentials (sometimes)




**Tools used:**

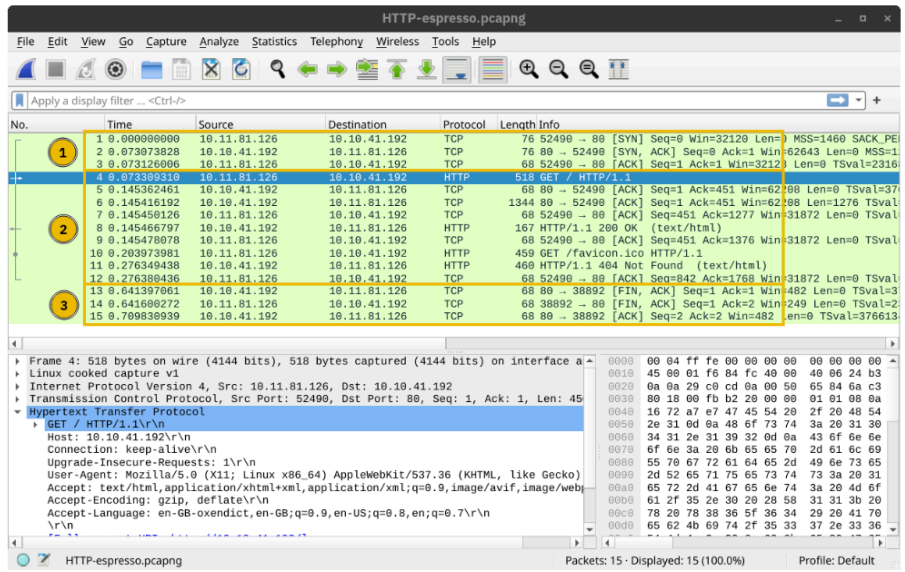
- Wireshark
- Tcpdump

## HTTP Communication

**Step 1 — DNS Resolution**
example.com → IP address

**Step 2 — TCP Three-Way Handshake**
The client establishes a TCP connection.
Packets for the handshake are marked as 1

SYN
SYN-ACK
ACK




**Step 3 — HTTP Communication**
Data communication packets are marked as 2 in the screenshot

Example request:

```
GET / HTTP/1.1
```

Server sends:
HTTP/1.1 200 OK

**Step 4 — TCP Connection Termination**

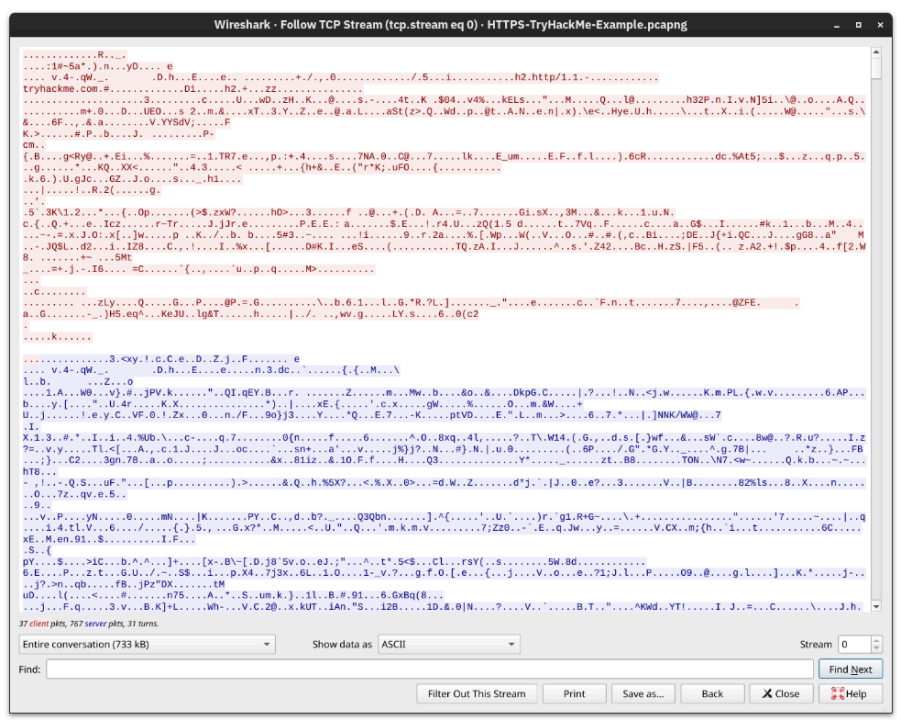
Connection closes using:
FIN
ACK
FIN
ACK

**HTTPS (HTTP Over TLS)**
HTTPS = HTTP + TLS encryption

| | |
|---|---|
| Transport protocol | TCP |
| Port | 443 |
| Encryption | TLS |

**HTTPS protects:**

- Login credentials
- Payment data
- Messages
- Personal information



## HTTPS Communication

**Step 1 — TCP Handshake**
Marked as 1 in the screenshot
SYN
SYN-ACK
ACK

**Step 2 — TLS Handshake**
Client and server negotiate encryption.
In the screenshot above, 1 and 2 are where
the TLS negotiation and establishment take place.

Main tasks:

- Select encryption algorithm
- Exchange keys
- Authenticate server certificate

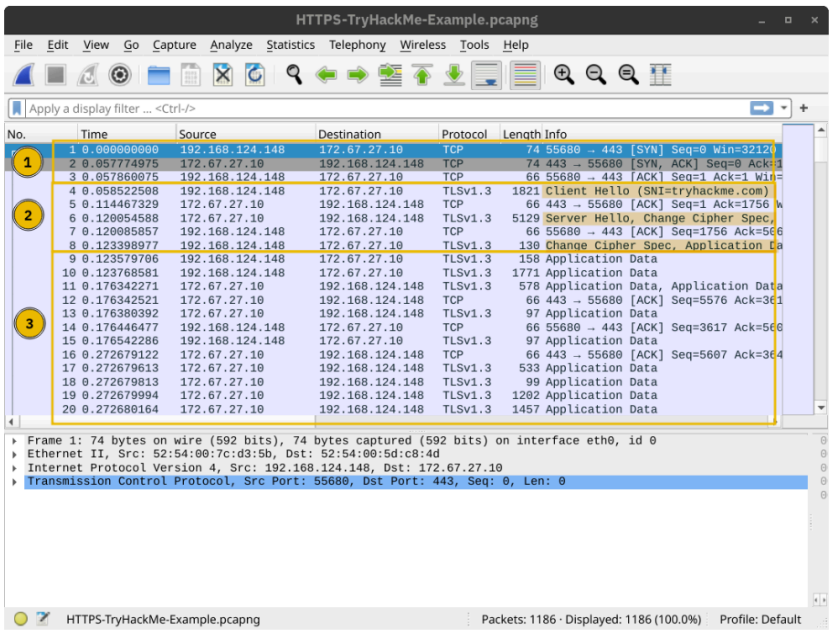
**Step 3 — Encrypted HTTP Communication**
Data communication packets are
marked as 3 in the screenshot.

After TLS is established:

```
GET /login HTTP/1.1
```

But in Wireshark it appears as:
TLS Application Data

because it is encrypted.



# TLS for protocols

TLS can be added to many protocols to provide secure communication.

| Insecure protocol | Secure version (TLS) |
|---|---|
| HTTP (80) | HTTPS (443) |
| SMTP (25) | SMTPS (465 and 587) |
| POP3 (110) | POP3S (995) |
| IMAP (143) | IMAPS (993) |

# SSH (Secure Shell)

**Older remote login protocol:**
TELNET: Sends data in plaintext

**Example of insecure login:**
user: admin
password: admin123

Anyone capturing network traffic using tools like Wireshark could read the credentials.
SSH came with the solution.

| Protocol | Port |
|---|---|
| TELNET | 23 |
| SSH | 22 |

## SSH (Secure Shell)

SSH encrypts communication between the client and server.

**SSH connection syntax:**

```
ssh username@hostname
```

```
ssh kali@192.168.1.10
```

### SSH Tunneling

SSH can create encrypted tunnels.

- Local computer → SSH tunnel → Remote server
- Other protocols can travel through the tunnel securely.
- This is similar to a VPN-like connection.

## SFTP (SSH File Transfer Protocol)

SFTP is part of the SSH protocol suite.
It provides secure file transfer over SSH encryption.

| | |
|---|---|
| Security | SSH encryption |
| Default Port | 22 |
| Protocol Base | SSH |

**Connection example:**

```
sftp username@hostname
```

**Common SFTP Commands**

| | |
|---|---|
| ls | list files |
| cd | change directory |
| get filename | download file |
| put filename | upload file |
| exit | close connection |

## FTPS (FTP Secure)

FTPS = FTP over TLS encryption.

FTP → FTPS

| | |
|---|---|
| Full Name | File Transfer Protocol Secure |
| Security | TLS / SSL |
| Default Port | 990 |
| Protocol Base | FTP |

FTPS requires a TLS certificate to encrypt connections.

# VPN (Virtual Private Network)

A VPN allows devices in different locations to communicate securely over the Internet as if they were on the same private network.

**TCP/IP provides:**

- Packet delivery
- Retransmission
- Routing around failures

**But no built-in protection for:**

- Confidentiality
- Integrity
- Privacy

VPN solves this by encrypting network traffic.

## VPN Security Issues

Even VPNs can have problems.

Examples:

- DNS Leaks
- Your DNS requests may bypass the VPN.

This can reveal:

- Your real location

Tests like DNS leak tests are used to detect this